24MCADMT201 ADVANCED DATABASE MANAGEMENT SYSTEMS

UNIT 2 :

Database Design:- Database Tables and Normalization – The Need for Normalization – The Normalization Process: Inference Rules for Functional Dependencies (proof not needed) - Minimal set of Functional Dependencies - Conversion to First Normal Form, Conversion to Second Normal Form, Conversion to Third Normal Form - Improving the Design - Higher Level Normal Forms: Boyce/Codd Normal Form, Fourth Normal Form, Join dependencies and Fifth Normal Form – Normalization and Database Design.

Database Design:- Database Tables and Normalization – The Need for Normalization

Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

- Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).
- Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistencies problems also arise during insert, delete, and update operations.
- In the relational model, there exist standard methods to quantify how efficient a databases is. These methods are called normal forms and there are algorithms to covert a given database into normal forms.
- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

Need of Normalization

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases. Normalization consists of a set of procedures that assist you in developing an effective database structure.

- **Insertion Anomalies**: Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains

information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.

- **Updation anomalies:** Updation anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment,Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing

Problems in the Employee_Department Relation

1. **Insertion Anomaly:**
   - If a new department is created but no employee is assigned to it yet, we cannot store its location because we need an employee record to insert.
2. **Update Anomaly:**
   - If the location of the HR department changes, we must update it in multiple rows (for both Nick Wise and Lily Case). If one row is missed, the data becomes inconsistent.
3. **Deletion Anomaly:**
   - If all employees in the IT department leave, we lose the department information, including its location.
4. **Data Redundancy:**
   - The department location is repeated for every employee in the same department.

After Normalization

Employee

Emp_ID	Emp_Name	Dept_ID
101	Nick Wise	D1
102	John Cader	D2
103	Lily Case	D1
104	Ford Dawid	D3

Department

Dept_ID	Department	Dept_Location
D1	HR	London
D2	Finance	Australia
D3	IT	Chicago

Employee_Skills

Emp_ID	Emp_Skills
101	Recruitment
101	Payroll
102	Budgeting
103	Recruitment
104	Programming
104	Testing

Before Normalization: The table is prone to redundancy and anomalies (insertion, update, and deletion).

After Normalization: The data is divided into logical tables to ensure consistency, avoid redundancy and remove anomalies making the database efficient and reliable.

Important Points of Normal Forms in DBMS

Purpose of Normal Forms:

To organize data efficiently, eliminate redundancy, and prevent anomalies during data operations like insertion, deletion and updates.

Types of Normal Forms

- **First Normal Form (1NF):** This is the most basic level of normalization. In 1NF, each table cell should contain only a single value, and each column should have a unique name. The first normal form helps to eliminate duplicate data and simplify queries.
- **Second Normal Form (2NF):** 2NF eliminates redundant data by requiring that each non-key attribute be dependent on the primary key. This means that each column should be directly related to the primary key, and not to other columns.
- **Third Normal Form (3NF):** 3NF builds on 2NF by requiring that all non-key attributes are independent of each other. This means that each column should be directly related to the primary key, and not to any other columns in the same table.
- **Boyce-Codd Normal Form (BCNF):** BCNF is a stricter form of 3NF that ensures that each determinant in a table is a candidate key. In other words, BCNF ensures that each non-key attribute is dependent only on the candidate key.
- **Fourth Normal Form (4NF):** 4NF is a further refinement of BCNF that ensures that a table does not contain any multi-valued dependencies.
- **Fifth Normal Form (5NF):** 5NF is the highest level of normalization and involves decomposing a table into smaller tables to remove data redundancy and improve data integrity.

Advantages of Normal Form

- **Reduced data redundancy:** Normalization helps to eliminate duplicate data in tables, reducing the amount of storage space needed and improving database efficiency.
- **Improved data consistency:** Normalization ensures that data is stored in a consistent and organized manner, reducing the risk of data inconsistencies and errors.
- **Simplified database design:** Normalization provides guidelines for organizing tables and data relationships, making it easier to design and maintain a database.
- **Improved query performance:** Normalized tables are typically easier to search and retrieve data from, resulting in faster query performance.
- **Easier database maintenance:** Normalization reduces the complexity of a database by breaking it down into smaller, more manageable tables, making it easier to add, modify, and delete data.

The Normalization Process: Inference Rules for Functional Dependencies

A functional dependency occurs when one attribute uniquely determines another attribute within a relation. It is a constraint that describes how attributes in a table relate to each other. If attribute A functionally determines attribute B we write this as the A→B. Functional dependencies are used to mathematically express relations among database entities and are very important to understanding advanced concepts in Relational Database Systems.

Example:

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

From the above table we can conclude some valid functional dependencies:

- roll_no → { name, dept_name, dept_building }→ Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency
- roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.
- dept_name → dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building
- More valid functional dependencies: roll_no → name, {roll_no, name} ⇢ {dept_name, dept_building}, etc.

Here are some invalid functional dependencies:

- name → dept_name Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- dept_building → dept_name There can be multiple departments in the same building. Example, in the above table departments ME and EC are in the same building B2, hence dept_building → dept_name is an invalid functional dependency.
- More invalid functional dependencies: name → roll_no, {name, dept_name} → roll_no, dept_building → roll_no, etc.

Key Inference Rules

1. **Reflexivity**: If Y is a subset of X, then X→Y
2. **Augmentation**: If X→Y then XZ→YZ for any attribute set Z.
3. **Transitivity**: If X→Y and Y→Z then X→Z.

Example

Initial Table

Consider a table of students with the following attributes:

StudentID	CourseID	Instructor	Department
1	101	Dr. Smith	Math

1	102	Dr. Johnson	Science
2	101	Dr. Smith	Math
3	103	Dr. Brown	Arts

Given Functional Dependencies

Let's define some functional dependencies based on the table:

1. **FD1**: StudentID, CourseID→Instructor (Each student in a course has one instructor.)
2. **FD2**: Instructor→Department (Each instructor belongs to one department.)

Using Inference Rules

1. **Reflexivity**:
    - From StudentID, CourseID→Instructor by reflexivity, we can say:
    - StudentID→StudentID [StudentID is a subset of itself, reinforces the idea that each StudentID is unique and unambiguous in the context of the database]
2. **Augmentation**:
    - From StudentID, CourseID→Instructor we can add an attribute Z(e.g., a new attribute like Semester):
    - StudentID, CourseID, Semester→Instructor, Semester
3. **Transitivity**:
    - Since we have:
        - Instructor→Department
        - and StudentID, CourseID→Instructor
    - We can conclude:
        - StudentID, CourseID→Department

Using these inference rules, we can derive functional dependencies that help us understand the relationships between attributes. This is essential for the normalization process as it allows us to identify and eliminate redundancy, ensuring that our database schema is efficient and maintains data integrity.

Minimal Set of Functional Dependencies

A minimal set of functional dependencies is a set of FDs that satisfies the following conditions:

1. **No Redundant Dependencies**: Removing any FD from the set changes the closure of the set (i.e., the set does not imply all the same attributes).
2. **Irredundancy**: Each FD is essential for the closure, meaning it cannot be derived from other FDs in the set.

3. **Singleton Attributes**: The right-hand side of each FD consists of a single attribute.

Example

Initial Table

Consider a table of students:

StudentID	CourseID	Instructor	Department
1	101	Dr. Smith	Math
1	102	Dr. Johnson	Science
2	101	Dr. Smith	Math
3	103	Dr. Brown	Arts

Functional Dependencies

Let's define some functional dependencies based on the table:

1. StudentID→Instructor
2. Instructor→Department
3. StudentID, CourseID→Instructor

Minimal Set of Functional Dependencies

Now, let's examine these FDs to identify a minimal set:

1. **Remove Redundancies**:
    - We can see that StudentID→Instructor is not necessary because StudentID, CourseID→Instructor covers it.Thus, it can be removed.
2. **Check for Irredundancy**:
    - We check Instructor→Department as essential because it cannot be derived from the other dependencies.
3. **Final Minimal Set**:
    - The minimal set of functional dependencies is:
        - StudentID, CourseID→Instructor
        - Instructor→Department

The minimal set of functional dependencies for the given table is:

1. StudentID, CourseID→Instructor
2. Instructor→Department

This set is essential for defining the relationships in the database without any redundancy, making it efficient for the normalization process.

First Normal Form (1NF)

First Normal Form (1NF) is the most basic level of normalization in a DBMS. The rules for achieving 1NF are as follows:

1. Each table should have a primary key, which uniquely identifies each record in the table.
2. Each column in the table should contain only atomic values, which means that a single cell should contain a single value and not a list of values.
3. There should be no repeating groups of data.

For example, let's say we have a table named "Students" that stores information about students in a school. A table that is not in 1NF might look like this:

StudentID	Name	Subject	Grade
1	John Smith	Math, Science	A
2	Jane Doe	English, History	B

In this table, the Subject column contains multiple values separated by commas, which violates the rule of atomic values. This table is not in 1NF. To bring this table to 1NF, we would need to split the Subject column into multiple columns, one for each subject, and repeat the student information for each subject taken.

StudentID	Name	Subject	Grade
1	John Smith	Math	A
1	John Smith	Science	A
2	Jane Doe	English	B
2	Jane Doe	History	B

This table is now in 1NF, since all the data is atomic, each cell contains only one value, and there are no repeating groups of data.

1NF is the starting point for normalization, and to ensure the data integrity and consistency it's necessary to move to next normalization forms.

Second Normal Form (2NF)

Second Normal Form (2NF) builds upon the rules of First Normal Form (1NF) by addressing the issue of partial dependencies. In 2NF, a table must not have any partial dependencies. A partial dependency exists when a non-primary key column depends on only part of a composite primary key.

For example, let's say we have a table named "Orders" that stores information about customer orders. A table that is not in 2NF might look like this:

OrderID	CustomerID	Product	Quantity	Price
1	1	A	2	10
2	1	B	1	20
3	2	A	3	10

In this table, the Price column depends on the Product column and the primary key is composed of OrderID and CustomerID. The table is not in 2NF because the Price column is functionally dependent on only part of the primary key (Product) and not on the whole primary key (OrderID and CustomerID).

To bring this table to 2NF, we need to separate the table into two separate tables: one for the Orders and one for the Products.

Table: Orders

OrderID	CustomerID	Product	Quantity
1	1	A	2
2	1	B	1
3	2	A	3

Table: Products

Product	Price
A	10
B	20

Now, the Price column is dependent on the primary key of the Products table (Product) and the Orders table has no partial dependencies. This design is now in 2NF.

2NF eliminates partial dependencies and improves the data integrity by reducing the data anomalies. However, it's not enough to ensure the data consistency and to avoid data anomalies, so it's necessary to move to the next normalization forms.

Third Normal Form (3NF)

Third Normal Form (3NF) builds upon the rules of Second Normal Form (2NF) by addressing the issue of transitive dependencies. In 3NF, a table must not have any transitive dependencies. A transitive dependency exists when a non-primary key column depends on another non-primary key column, rather than on the primary key.

To achieve 3NF, a table must already be in 2NF and all non-primary key columns must be directly dependent on the primary key.

For example, let's say we have a table named "Employees" that stores information about employees in a company. A table that is not in 3NF might look like this:

EmployeeID	**Name**	**Department**	**Manager**
1	John	IT	Tom
2	Jane	HR	Tom
3	Tom	Management	

In this table, the Manager column depends on the Department column, and neither of them depend on the primary key (EmployeeID). This table is not in 3NF because of the transitive dependency between the Manager column and the Department column.

To bring this table to 3NF, we need to separate the table into two separate tables: one for the Employees and one for the Departments.

Table: Employees

EmployeeID	**Name**	**Department**
1	John	IT
2	Jane	HR
3	Tom	Management

Table: Departments

Department	Manager
IT	Tom
HR	Tom
Management	

Now, the Manager column is dependent on the primary key of the Departments table (Department) and the Employees table has no transitive dependencies. This design is now in 3NF.

3NF eliminates transitive dependencies and improves the data integrity and consistency by reducing the data anomalies. However, it's still not enough to ensure the data consistency and to avoid data anomalies, so it's necessary to move to the next normalization forms like Boyce-Codd Normal Form (BCNF) or Fourth Normal Form (4NF) in some cases.

Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF) is a higher level of normalization than Third Normal Form (3NF). It addresses the issue of non-trivial functional dependencies. A functional dependency is said to be non-trivial if it doesn't hold true for the case where the left-hand side is a subset of the primary key.

A table is in BCNF if and only if for every non-trivial functional dependency X → Y, X is a superkey. A superkey is a set of columns that can uniquely identify a row in a table.

For example, let's say we have a table named "Invoices" that stores information about invoices of a company. A table that is not in BCNF might look like this:

InvoiceNumber	CustomerID	Product	Quantity	Price
1	1	A	2	10
2	1	B	1	20
3	2	A	3	10

In this table, the primary key is (InvoiceNumber) and it has a non-trivial functional dependency CustomerID → Product. This dependency doesn't hold true for the case where the left-hand side is a subset of the primary key (InvoiceNumber) because the product can be different for the same customer. So this table is not in BCNF.

To bring this table to BCNF, we need to separate the table into two separate tables: one for the Invoices and one for the Customer_Product.

Table: Invoices

InvoiceNumber	**Quantity**	**Price**
1	2	10
2	1	20
3	3	10

Table: Customer_Product

CustomerID	**Product**
1	A
1	B
2	A

Now, the table Invoices has the primary key (InvoiceNumber) and the table Customer_Product has the primary key (CustomerID, Product) . This design is now in BCNF.

BCNF is considered to be the strongest normal form and it eliminates non-trivial functional dependencies and improves the data integrity and consistency by reducing the data anomalies. However, it's not necessary to go through BCNF before going to higher normal forms like fourth normal form (4NF) or fifth normal form (5NF).

Fourth Normal Form (4NF)

Fourth Normal Form (4NF) is a higher level of normalization than Boyce-Codd Normal Form (BCNF). It addresses the issue of multi-valued dependencies. A multi-valued dependency is a type of dependency where a non-primary key column is functionally dependent on two or more independent non-primary key columns.

A table is in 4NF if and only if it does not contain any multi-valued dependencies.

For example, let's say we have a table named "Orders" that stores information about orders of a company. A table that is not in 4NF might look like this:

OrderNumber	Customer	Product	Supplier
1	John	A	S1
2	Jane	B	S2
3	Tom	A	S1

In this table, there is a multi-valued dependency between Product and Supplier. This dependency doesn't hold true for the case where a product is supplied by multiple suppliers or a supplier supplies multiple products. So this table is not in 4NF.

To bring this table to 4NF, we need to separate the table into two separate tables: one for the Orders and one for the Product_Supplier.

Table: Orders

OrderNumber	Customer	Product
1	John	A
2	Jane	B
3	Tom	A

Table: Product_Supplier

Product	Supplier
A	S1
B	S2

Now, the table Orders has no multi-valued dependencies and the table Product_Supplier has primary key (Product, Supplier) . This design is now in 4NF.

4NF eliminates multi-valued dependencies and improves the data integrity and consistency by reducing the data anomalies. However, it's not necessary to go through 4NF before going to higher normal forms like fifth normal form (5NF).

Fifth Normal Form (5NF)

Fifth Normal Form (5NF), also known as Projection-Join Normal Form (PJ/NF) is a higher level of normalization than Fourth Normal Form (4NF). It addresses the issue of join dependency. A join dependency is a type of dependency where a table can be split into two or more tables, each of which contains a subset of the original table's columns, and can be recombined by joining the tables on their common columns.

A table is in 5NF if and only if it does not contain any join dependencies.

For example, let's say we have a table named "Employee_Department" that stores information about employees and their departments. A table that is not in 5NF might look like this:

EmployeeID	EmployeeName	Department	Salary
1	John	IT	50000
2	Jane	HR	40000
3	Tom	IT	60000

In this table, there is a join dependency between Employee and Department. This dependency doesn't hold true for the case where an employee can belong to multiple departments or a department can have multiple employees. So this table is not in 5NF.

To bring this table to 5NF, we need to separate the table into three separate tables: one for the Employee, one for the Department, and one for the Employee_Department_Salary.

Table: Employee

EmployeeID	EmployeeName
1	John
2	Jane
3	Tom

Table: Department

Department
IT
HR

Table: Employee_Department_Salary

EmployeeID	Department	Salary
1	IT	50000
2	HR	40000
3	IT	60000

Now, the tables Employee, Department, and Employee_Department_Salary has no join dependency and each table has its own primary key. This design is now in 5NF.
5NF eliminates join dependencies and improves the data integrity and consistency by reducing the data anomalies.

Join Dependencies

A **Join Dependency** occurs when a relation can be recreated by joining multiple projections of that relation. This is important in the context of database normalization to ensure that all data can be reconstructed without any loss.

Fifth Normal Form (5NF)

A relation is in **Fifth Normal Form (5NF)** if it is in Fourth Normal Form (4NF) and every join dependency in the relation is a consequence of the candidate keys. This means that the relation cannot be decomposed into smaller relations without losing data.

Example

Consider a relation **R**:

StudentID	Course	Instructor
1	Math	Dr. Smith
1	Science	Dr. Johnson
2	Math	Dr. Smith
2	Science	Dr. Johnson
3	Math	Dr. Brown

Join Dependency in R

Here, we can see that:

- A student can enroll in multiple courses.
- Each course can be taught by multiple instructors.

This creates a join dependency because we can decompose the relation into smaller relations based on the combinations of `StudentID`, `Course`, and `Instructor`.

Decomposing into 5NF

To achieve 5NF, we can decompose the relation **R** into three separate relations:

1. **Student-Course Relation**:

StudentID	Course
1	Math
1	Science
2	Math
2	Science
3	Math

2. **Course-Instructor Relation**:

Course	Instructor
Math	Dr. Smith
Science	Dr. Johnson
Math	Dr. Brown

3. **Student-Instructor Relation** (if needed):

StudentID	Instructor
1	Dr. Smith
1	Dr. Johnson
2	Dr. Smith
2	Dr. Johnson
3	Dr. Brown

By decomposing the original relation into smaller relations, we ensure that:

1. Each relation is free from redundancy.
2. The relations are in the Fifth Normal Form, as all join dependencies are a consequence of the candidate keys.